Flexbox: The Game Changer

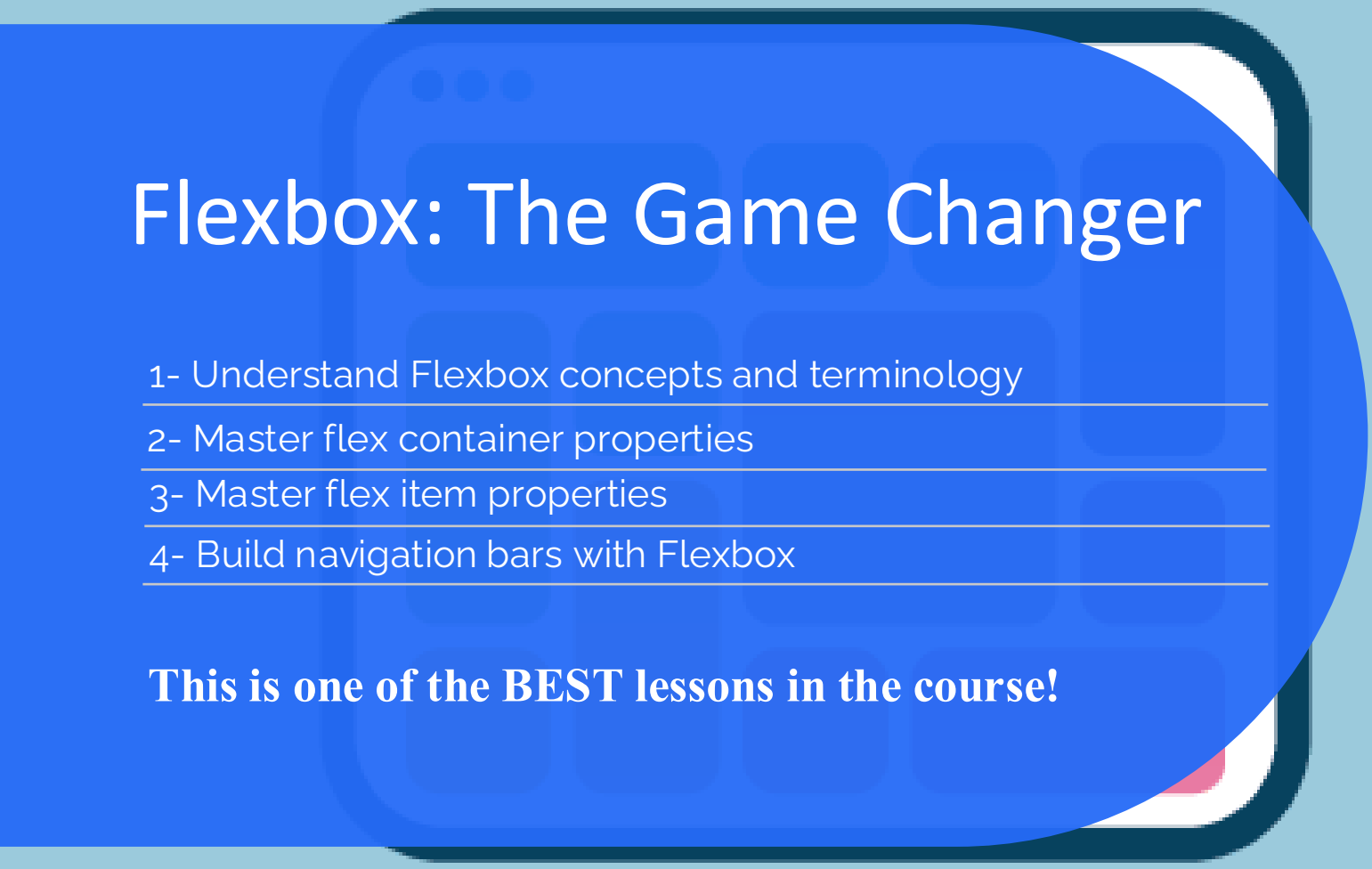
1- Understand Flexbox concepts and terminology

2- Master flex container properties

3- Master flex item properties

4- Build navigation bars with Flexbox

This is one of the BEST lessons in the course!



SCROLL

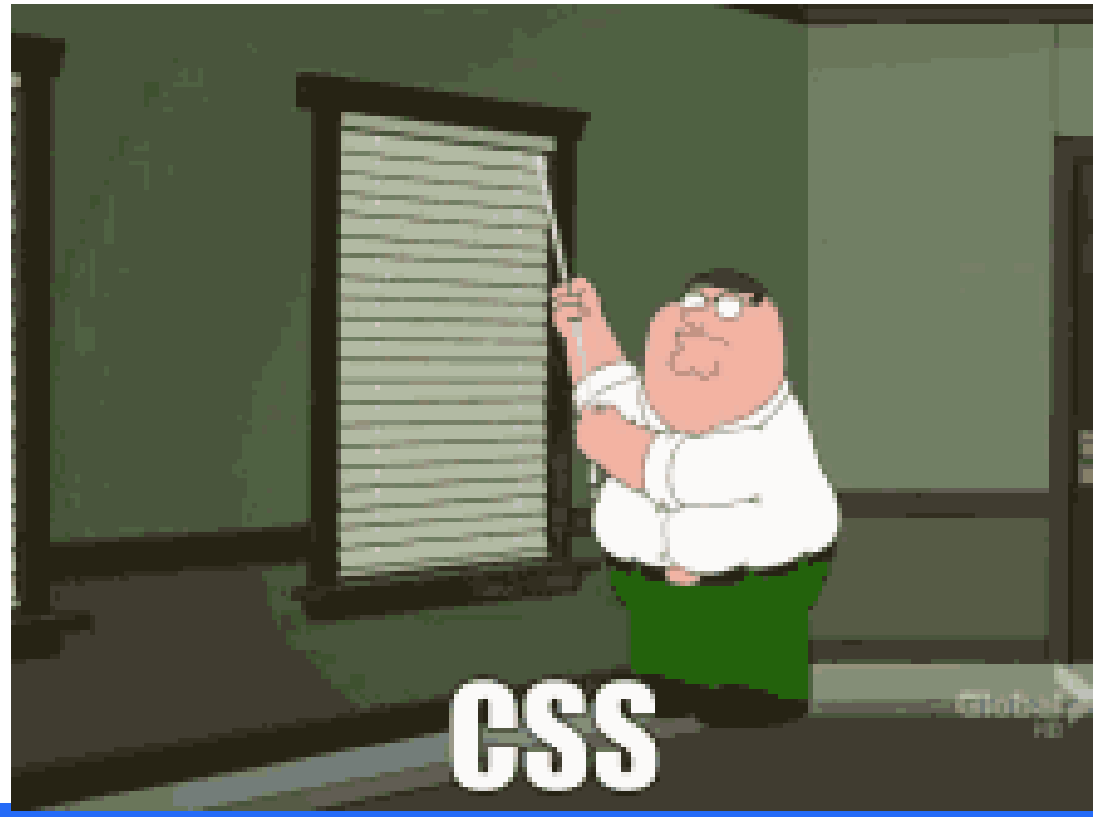
The Problem We Had

The Old Nightmare Way:

```css
.center {
  position: absolute;
  top: 50%;
  left: 50%;
  transform: translate(-50%, -50%);
}
```

What's Wrong With This?

- Confusing math (50% of what?!)
- Requires positioning knowledge
- Breaks document flow
- Hard to remember
- NOT intuitive at all



Developers literally googled "how to center a div" millions of times!

With Flexbox: Magic

What Changed?

- 3 lines instead of complicated transforms
- Readable - you can tell what it does!
- Intuitive - "justify content center" makes sense!
- Stays in document flow - no positioning hacks
- Works perfectly every single time

```css
.container {
  display: flex;
  justify-content: center; /* Center horizontally */
  align-items: center;    /* Center vertically */
  height: 100vh;          /* Full viewport height */
}
```

THAT'S why Flexbox revolutionized web design!



What Is Flexbox?

The Basics:

- **Released:** ~2015 (CSS3 specification)
- **Browser Support:** Excellent! (All modern browsers)
- **Purpose:** Make layout patterns intuitive and powerful

Key Feature: One-Dimensional Layout

- Controls **row OR column** (not both at once)
- Horizontal layouts (navigation bars, button groups)
- Vertical layouts (sidebars, stacked cards)
- *Need 2D? That's CSS Grid! (Next week)*

Why "Flexible Box"?

- Elements **flex** (grow/shrink) to fill available space
- **Flexible** sizing - adapts automatically
- Think: rubber bands that stretch and contract!

After Flexbox:

- Intuitive property names
- Works as expected
- Layouts make sense!

It revolutionized CSS layout! This changed EVERYTHING!

When to Use Flexbox?

Navigation & Headers:
- Navigation bars (horizontal menu items)
- Header layouts (logo left, links center, button right)
- Breadcrumbs and pagination
- Tab bars and toolbar buttons

Content Layouts:
- Card layouts (row of equal-height cards)
- Image galleries (horizontal scrolling)
- Social media feeds (Instagram stories layout)
- Product listings (items in a row)

NOT for:
- Complex 2D grids (rows AND columns at once)
- Magazine-style layouts with varying sizes
- → *Use CSS Grid for those! (Next week)*

UI Components:
- Button groups (side-by-side buttons)
- Form layouts (label + input on same line)
- Alert/notification bars
- Footer layouts (links distributed across)

Alignment Tasks:
- Centering content (vertically, horizontally, or both!)
- Distributing space evenly
- Aligning items to edges
- Any items that need to line up in a row OR column

Rule of Thumb:
Row OR Column = Flexbox;
Rows AND Columns = Grid

The Two Players

HTML:

```
<section class="container">
    <div class="box">1</div>
    <div class="box">2</div>
    <div class="box">3</div>
    <div class="box">4</div>
    <div class="box">5</div>
    <div class="box">6</div>
</section>
```

CSS:

```
.container {
  display: flex; /* Enables flexbox */
}
```

Flexbox Has Two Parts: Parent & Children

Flex Container (Parent)

- The element with `display: flex`
- Controls the overall layout
- Sets rules for how children behave
- The "stage" where items perform

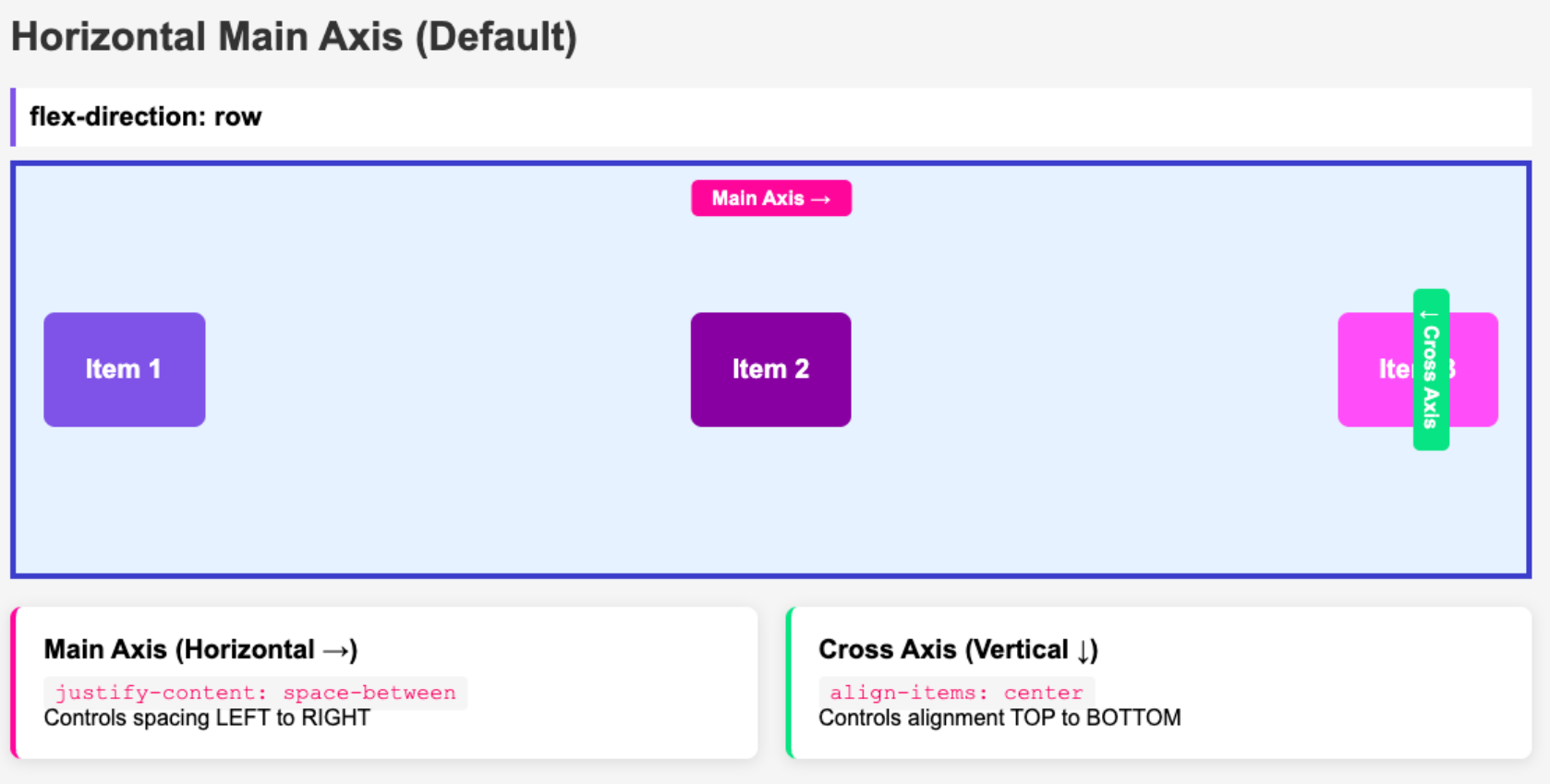
Flex Items (Children)

- **Direct children** automatically become flex items
- Follow parent's rules
- Can be controlled individually
- The "performers" on the stage

Key Rule: Only **DIRECT children** become flex items!

- Grandchildren? Not flex items (unless their parent is also flex)

Flexbox Axes: Horizontal Main Axis (Default)



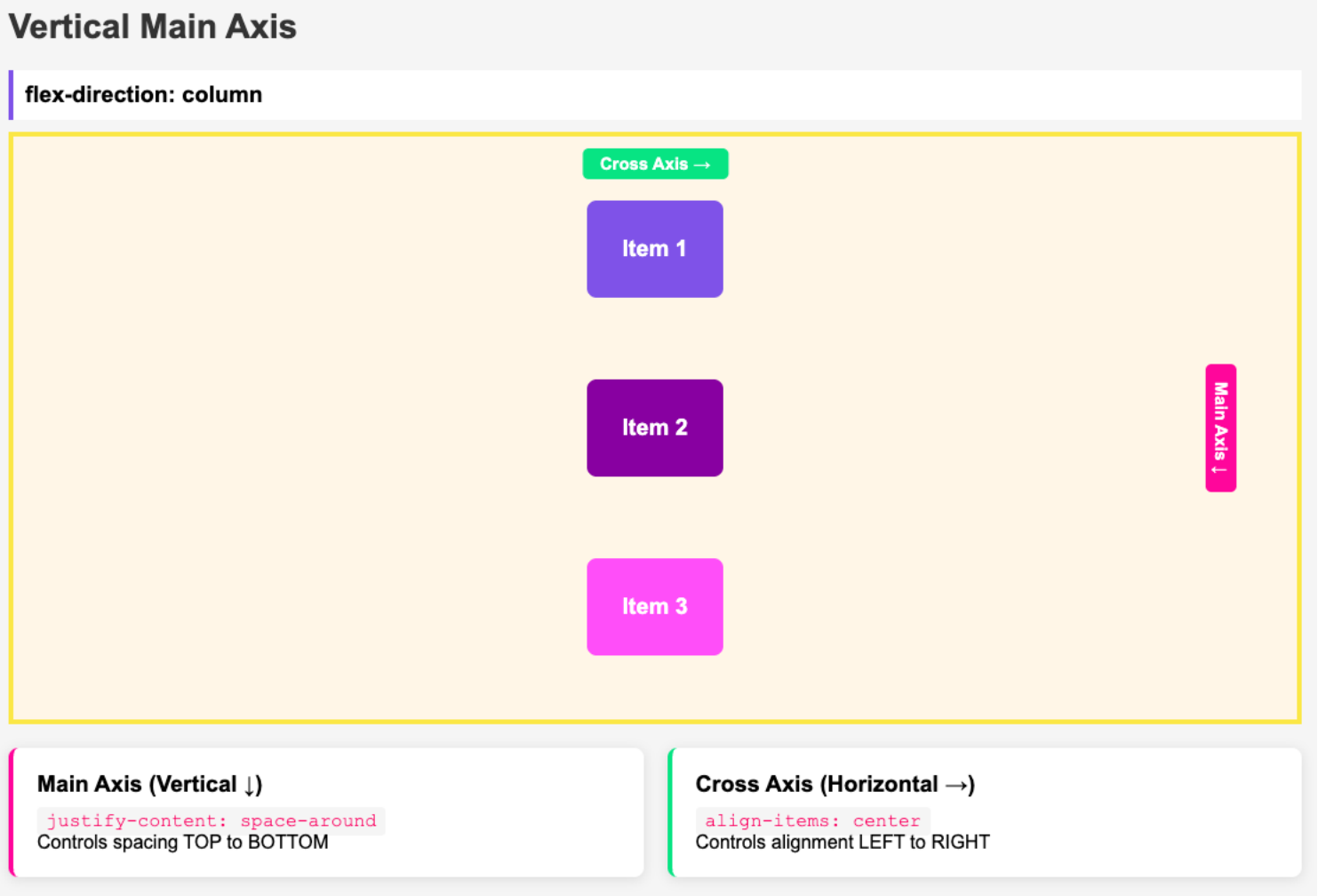
Main Axis → Primary direction of flex items

- **Default:** Horizontal (left to right)
- Can be changed to vertical with `flex-direction`
- Items line up ALONG the main axis

Why This Matters:

- justify-content → Controls spacing on **MAIN** axis
- align-items → Controls alignment on **CROSS** axis

Flexbox Axes: Vertical Main Axis



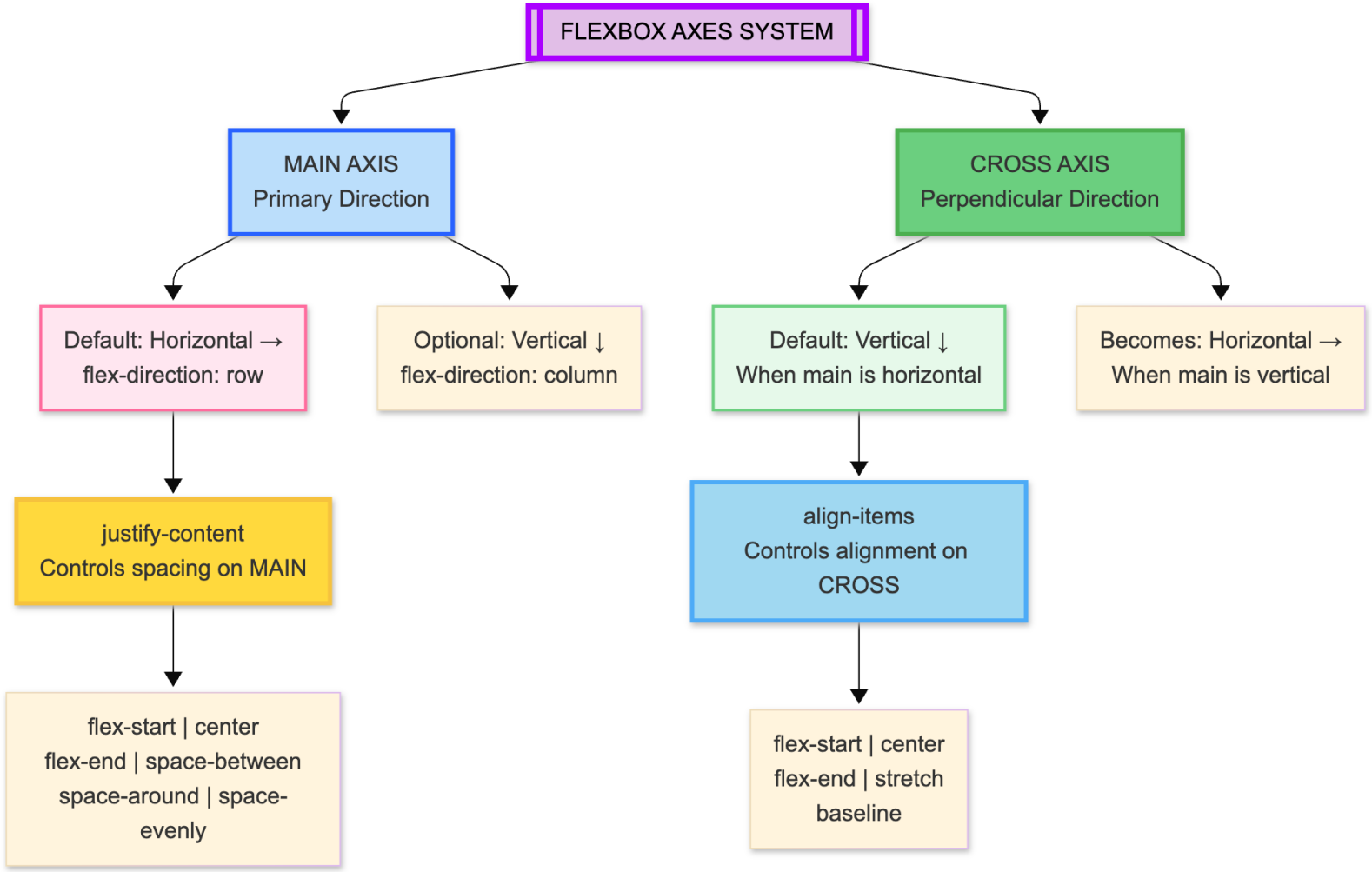
Cross Axis → Perpendicular to main axis

- **Default:** Vertical (top to bottom)
- Always 90° to the main axis
- Items stretch/align ACROSS the cross axis

Why This Matters:

- justify-content → Controls spacing on **MAIN** axis
- align-items → Controls alignment on **CROSS** axis

Flexbox Axes: Vertical Main Axis



Main Axis → Primary direction of flex items

- **Default:** Horizontal (left to right)
- Can be changed to vertical with `flex-direction`
- Items line up ALONG the main axis

Cross Axis → Perpendicular to main axis

- **Default:** Vertical (top to bottom)
- Always 90° to the main axis
- Items stretch/align ACROSS the cross axis

Why This Matters:

- justify-content → Controls spacing on **MAIN** axis
- align-items → Controls alignment on **CROSS** axis

CONTAINER PROPERTIES



Container Properties Overview

flex-direction
- Changes which way items flow (row or column)
- Controls the main axis direction

justify-content
- Distributes space on the MAIN axis
- "How should items be justified/spaced?"

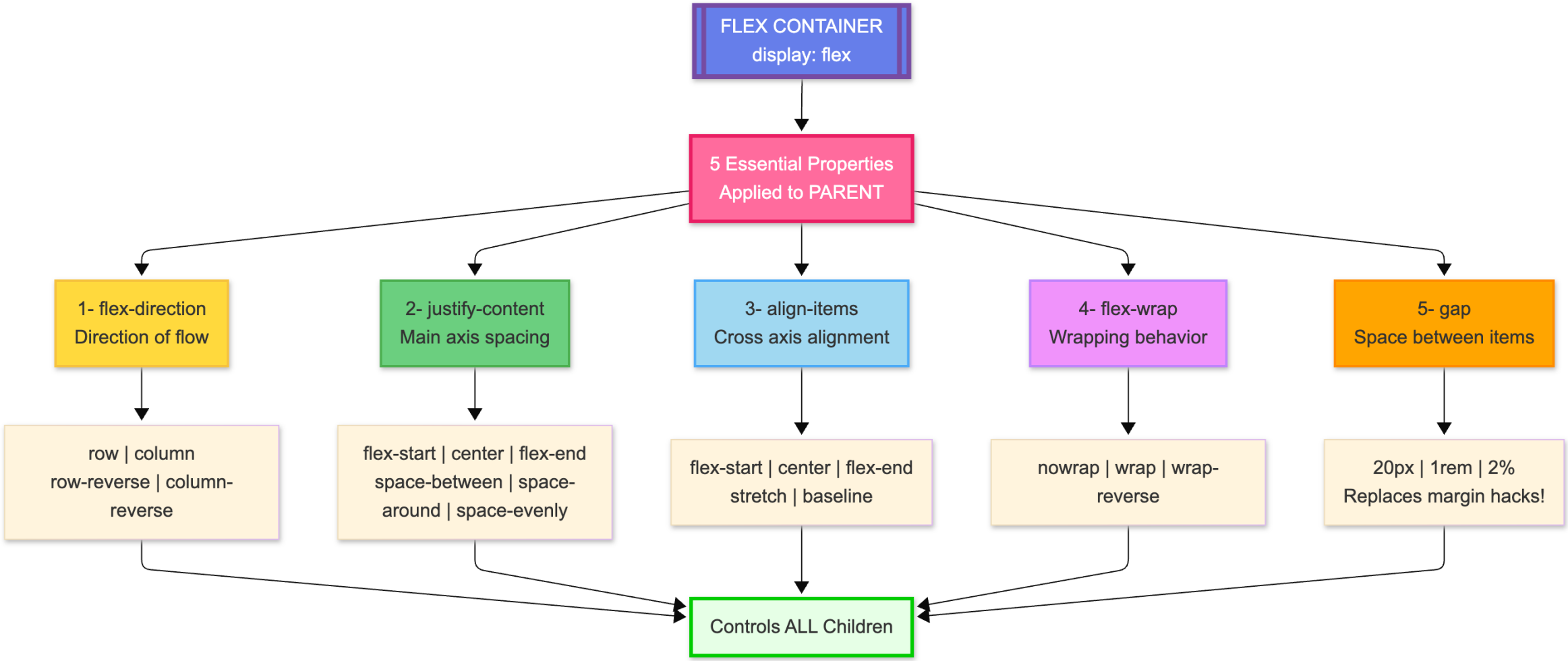
align-items
- Aligns items on the CROSS axis
- "How should items be aligned?"

flex-wrap
- Should items wrap to new lines?
- Makes layouts responsive automatically

gap
- Space BETWEEN items (modern & clean!)
- Replaces margin hacks

These control how ALL children behave!

Container Properties Overview



Flex-direction

flex-direction: Changes the Main Axis!

What This Changes:

- **Main axis direction** rotates!
- Row → Main axis is horizontal
- Column → Main axis is vertical

When to Use:

- `row`: Navigation bars, button groups (most layouts!)
- `column`: Vertical menus, sidebars, mobile layouts
- `*-reverse`: RTL languages, creative effects

```
The Four Directions:

row (default) ➡️
[1] [2] [3]  → Left to right (most common!)

row-reverse ⬅️
[3] [2] [1]  ← Right to left (reverse order!)

column ⬇️
[1]
[2]  ↓ Top to bottom (vertical!)
[3]

column-reverse ⬆️
[3]
[2]  ↑ Bottom to top (upside down!)
[3]
```

```css
.container {
  flex-direction: row;            /* → default */
  flex-direction: row-reverse;    /* ← */
  flex-direction: column;         /* ↓ */
  flex-direction: column-reverse; /* ↑ */
}
```

Flex Direction (row; row-reverse)

row (Default)

Left to right → Most common!

```
flex-direction: row; /* → default */
```



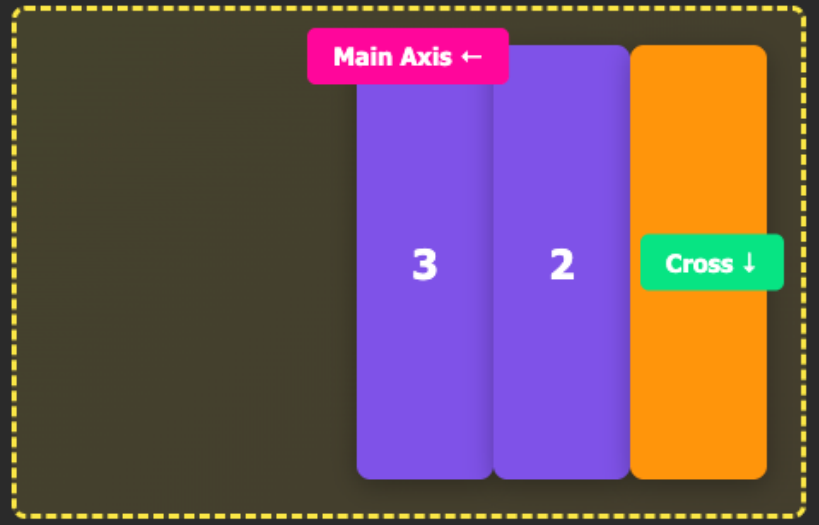
When to Use:

- ✓ Navigation bars
- ✓ Button groups
- ✓ Horizontal layouts
- ✓ Most common use case!

row-reverse

Right to left ← Reversed!

```
flex-direction: row-reverse; /* ← */
```



When to Use:

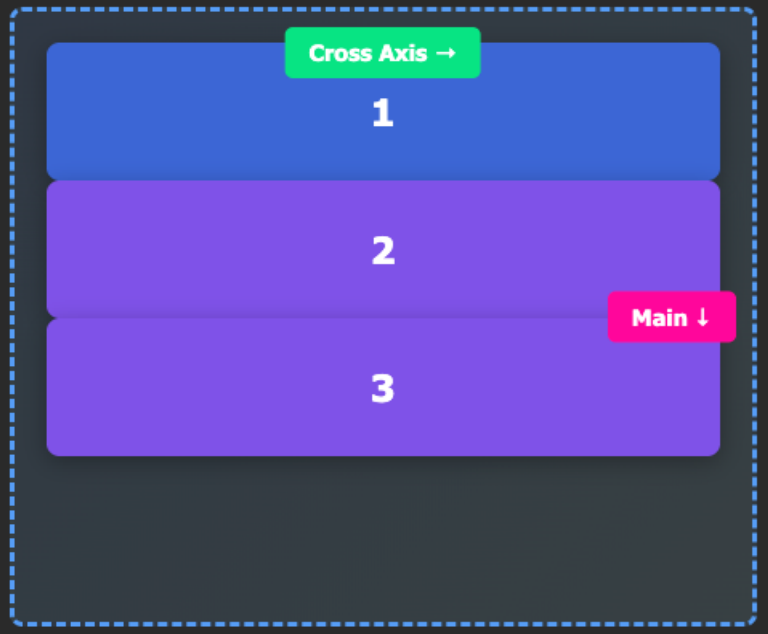
- ✓ RTL languages (Arabic, Hebrew)
- ✓ Creative effects
- ✓ Reverse chronological order

Flex Direction (column; column-reverse)

column

Top to bottom ↓ Vertical!

```
flex-direction: column; /* ↓ */
```



When to Use:

- ✓ Vertical menus
- ✓ Sidebars
- ✓ Mobile layouts
- ✓ Stacked forms

column-reverse

Bottom to top ↑ Upside down!

```
flex-direction: column-reverse; /* ↑ */
```



When to Use:

- ✓ Chat interfaces (newest at bottom)
- ✓ Comments sections
- ✓ Creative effects
- ✓ Sticky footers

Justify-content

justify-content: Main Axis Alignment!

Most Used:

- center - Centering content
- space-between - Navigation bars
- space-evenly - Even distribution

```
.container {
  justify-content: flex-start;     /* ■■■_____ */
  justify-content: flex-end;       /* _____■■■ */
  justify-content: center;         /* __■■■__ */
  justify-content: space-between;  /* ■___■___■ */
  justify-content: space-around;   /* _■__■__■_ */
  justify-content: space-evenly;   /* _■_■_■_ */
}
```

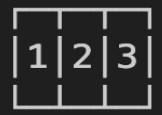
flex-start (default) – Start of main axis



flex-end – End of main axis



center – Middle of main axis (Very common!)



space-between – Max space between items



space-around – Equal space around each



space-evenly – Perfectly even spacing



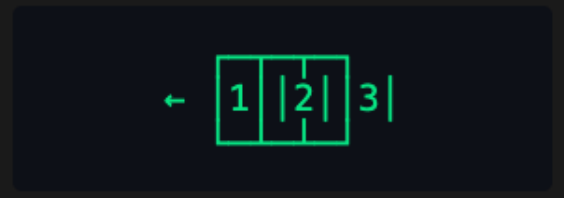
Justify-Content

flex-start



Items at **start**
Empty space at **end**

flex-end



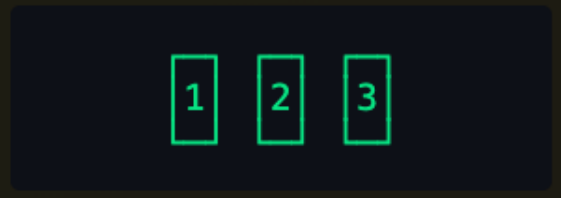
Items at **end**
Empty space at **start**

center



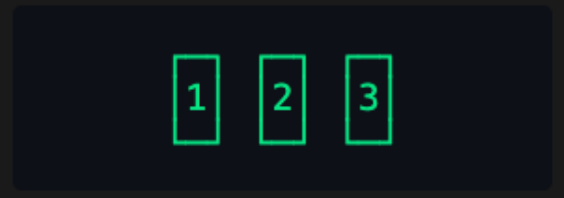
Items in **center**
Equal space **both sides**

space-between



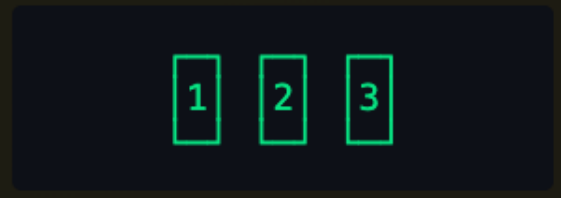
Max space between items
Edges **touch sides**

space-around



Equal space **around** each
Edge space = **half**

space-evenly

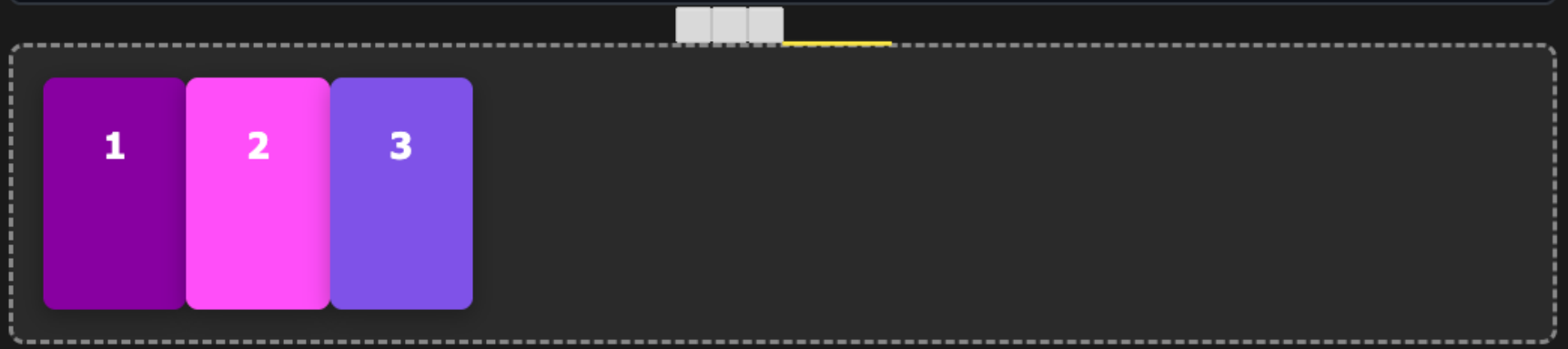


Perfectly even spacing
All gaps identical

Justify-Content (flex-start)

flex-start DEFAULT

```
justify-content: flex-start; /* Default */
```



When to Use:

- ✓ Default layouts
- ✓ Start-aligned content
- ✓ Standard left alignment

Justify-Content (flex-end)

flex-end

```
justify-content: flex-end;
```



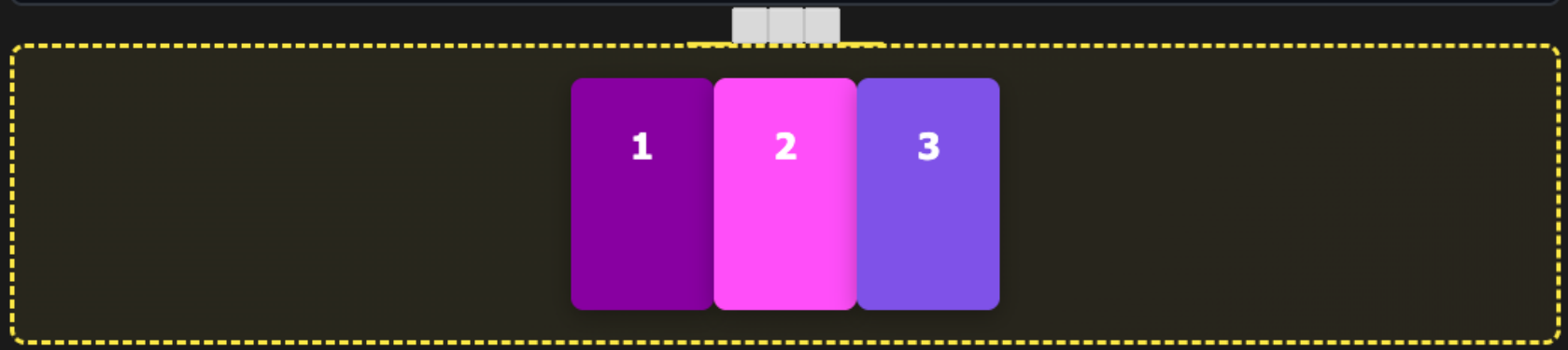
When to Use:

- ✓ Right-aligned navigation
- ✓ End-positioned content
- ✓ Profile icons, user menus

Justify-Content (center)

center MOST USED

```
justify-content: center; /* Very common! */
```



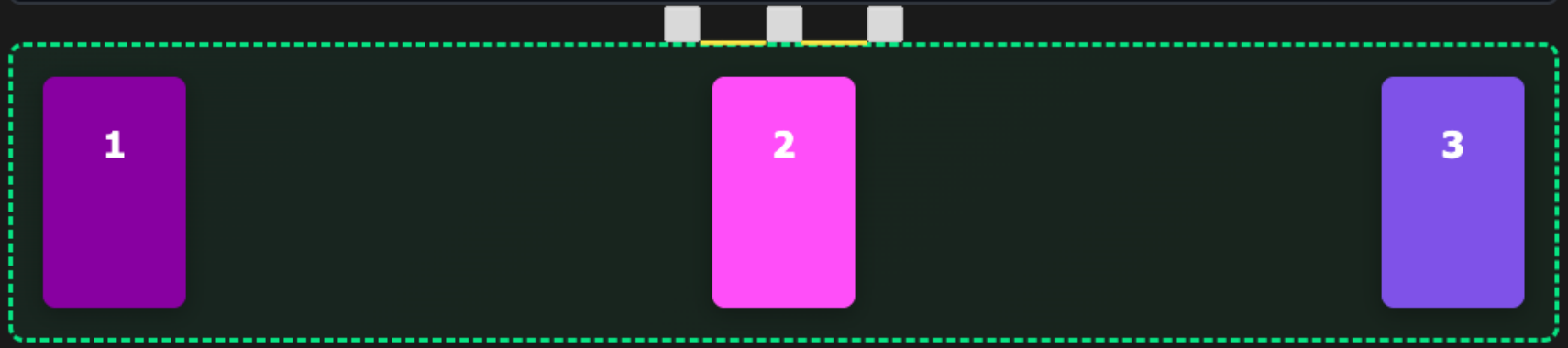
When to Use (VERY COMMON!):

- ✓ Hero sections & landing pages
- ✓ Centered content & CTAs
- ✓ Modal dialogs
- ✓ Button groups

Justify-Content (space-between)

space-between VERY COMMON

```
justify-content: space-between; /* Nav bars! */
```



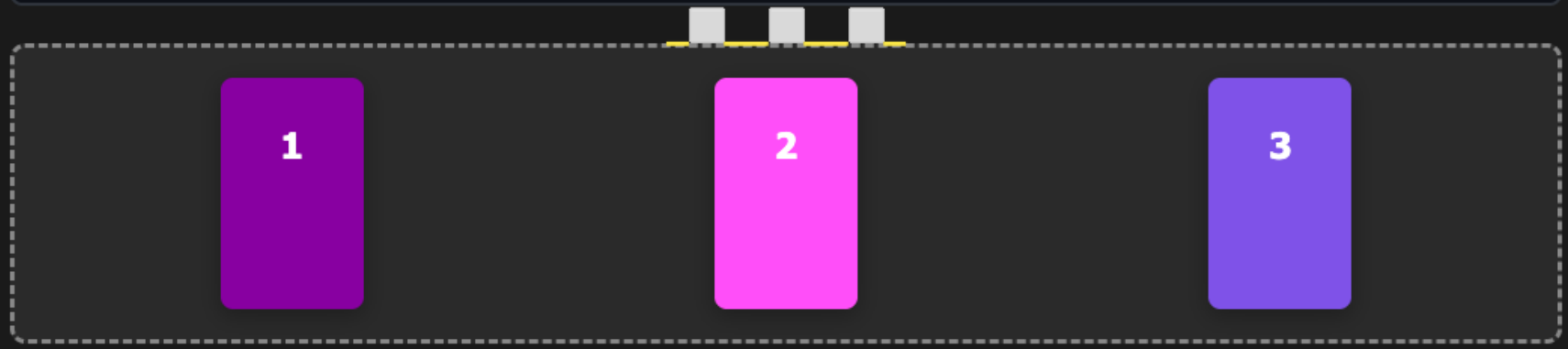
When to Use (VERY COMMON!):

- ✓ Navigation bars (logo + menu)
- ✓ Header layouts
- ✓ Footer sections
- ✓ Toolbars & controls

Justify-Content (space-around)

space-around

```
justify-content: space-around;
```



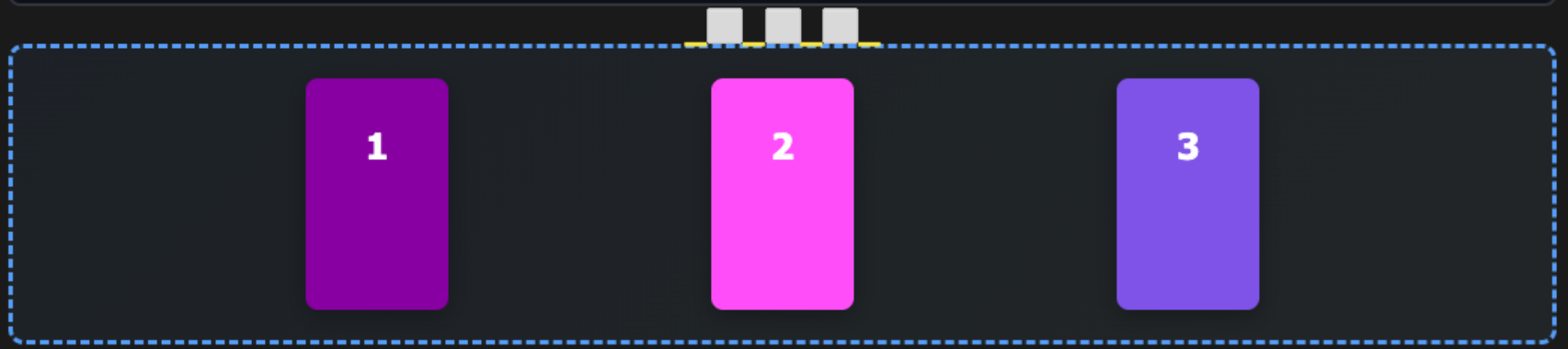
When to Use:

- ✓ Button groups
- ✓ Card layouts
- ✓ Icon sets
- ✓ (Older solution - prefer space-evenly)

Justify-Content (space-evenly)

space-evenly MODERN

```
justify-content: space-evenly; /* Perfect spacing! */
```



When to Use (MODERN!):

- ✓ Perfect distribution
- ✓ Gallery layouts
- ✓ Dashboard cards
- ✓ Feature sections

Align-items

align-items: Cross Axis Alignment!

Most Used:

- center - Centering content
- space-between - Navigation bars
- space-evenly - Even distribution

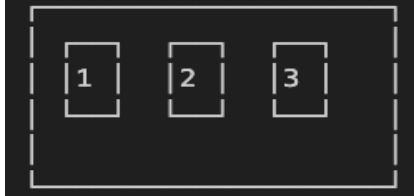
```
.container {
  align-items: stretch;    /* Fill height (default) */
  align-items: flex-start; /* Top */
  align-items: flex-end;   /* Bottom */
  align-items: center;     /* Middle! */
  align-items: baseline;   /* Text baseline */
}
```

stretch (default) – Fill container height



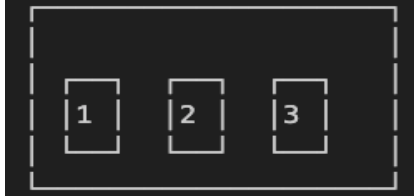
← All same height!

flex-start – Align to top (start of cross axis)



← Items at top

flex-end – Align to bottom (end of cross axis)



← Items at bottom

center – Vertical centering! THE SOLUTION!



← Perfectly centered!

baseline – Align text baselines (for typography)



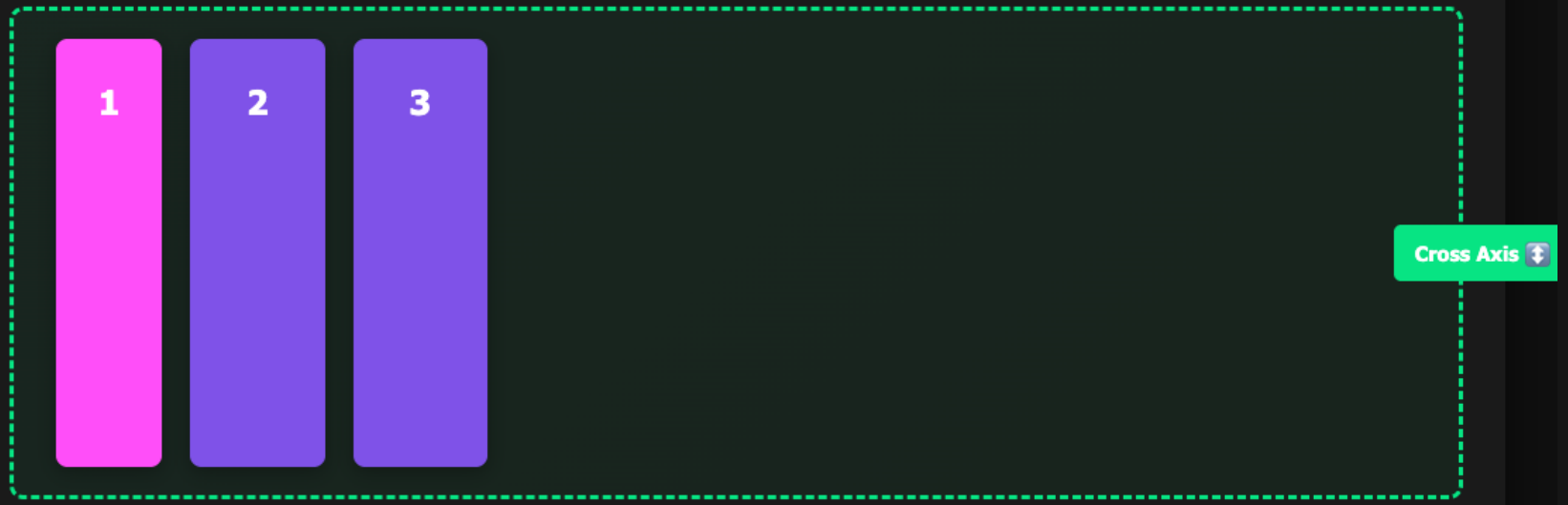
← Text lines up!

align-items (stretch)

stretch DEFAULT

```
align-items: stretch; /* Default - Fill height */
```

All Same Height!



When to Use:

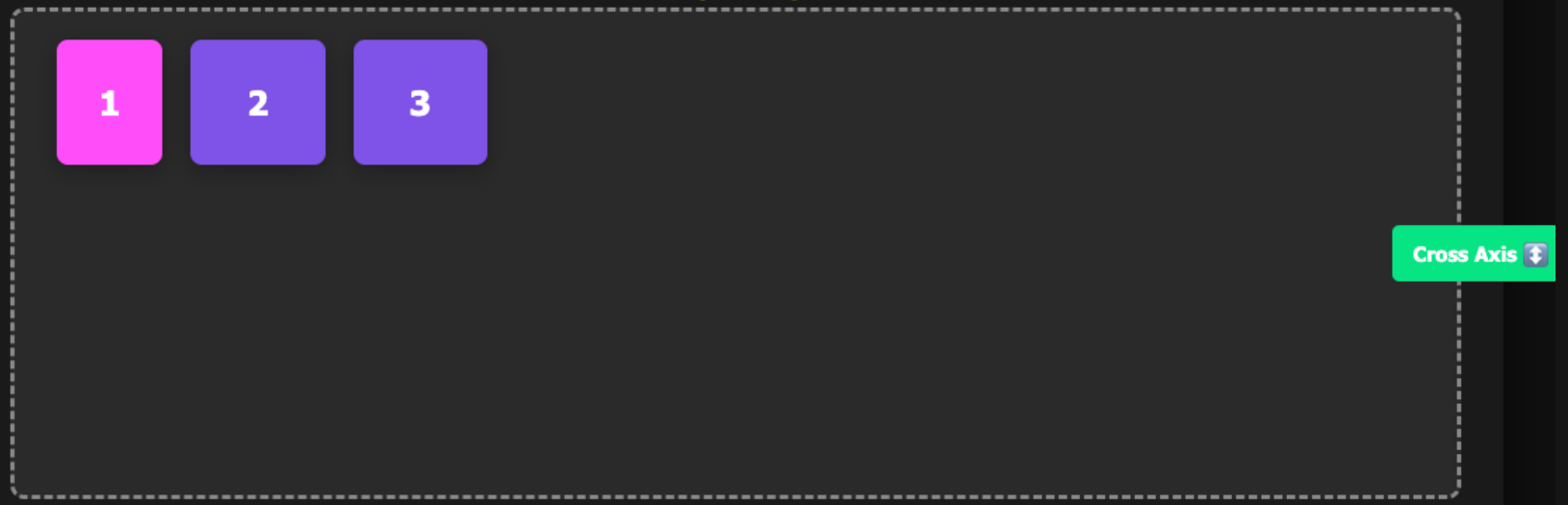
- ✓ Equal height cards (automatic!)
- ✓ Consistent column heights
- ✓ Navigation items
- ✓ Default behavior - items fill container

align-items (flex-start)

flex-start ⬆️

```
align-items: flex-start; /* Top alignment */
```

⬆️ Top Aligned



When to Use:

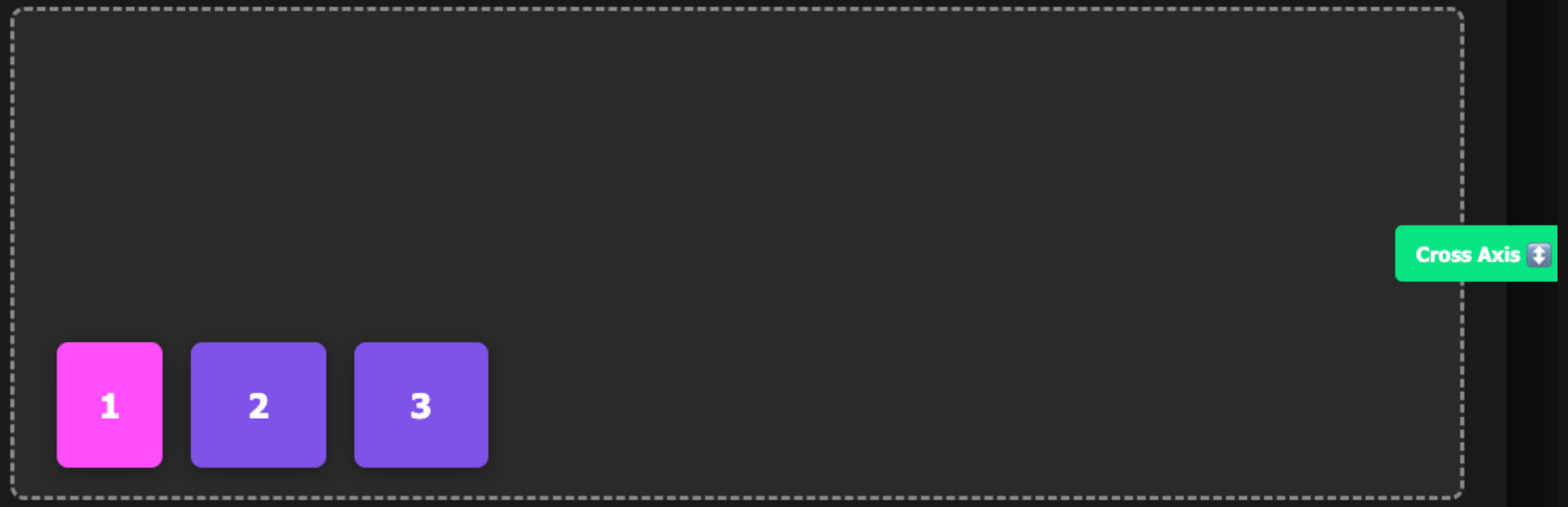
- ✓ Form labels
- ✓ Top-aligned content
- ✓ List items with varying heights
- ✓ Icons at top of cards

align-items (flex-end)

flex-end ⬇️

```
align-items: flex-end; /* Bottom alignment */
```

⬇️ Bottom Aligned



When to Use:

- ✓ Footer elements
- ✓ Bottom-aligned badges
- ✓ Status indicators at bottom
- ✓ Pricing button alignments

align-items (stretch)

center 🎯 ⭐ MOST USED

```
align-items: center; /* ⭐ Perfect vertical centering! */
```

🎯 Perfectly Centered!



When to Use (VERY COMMON!):

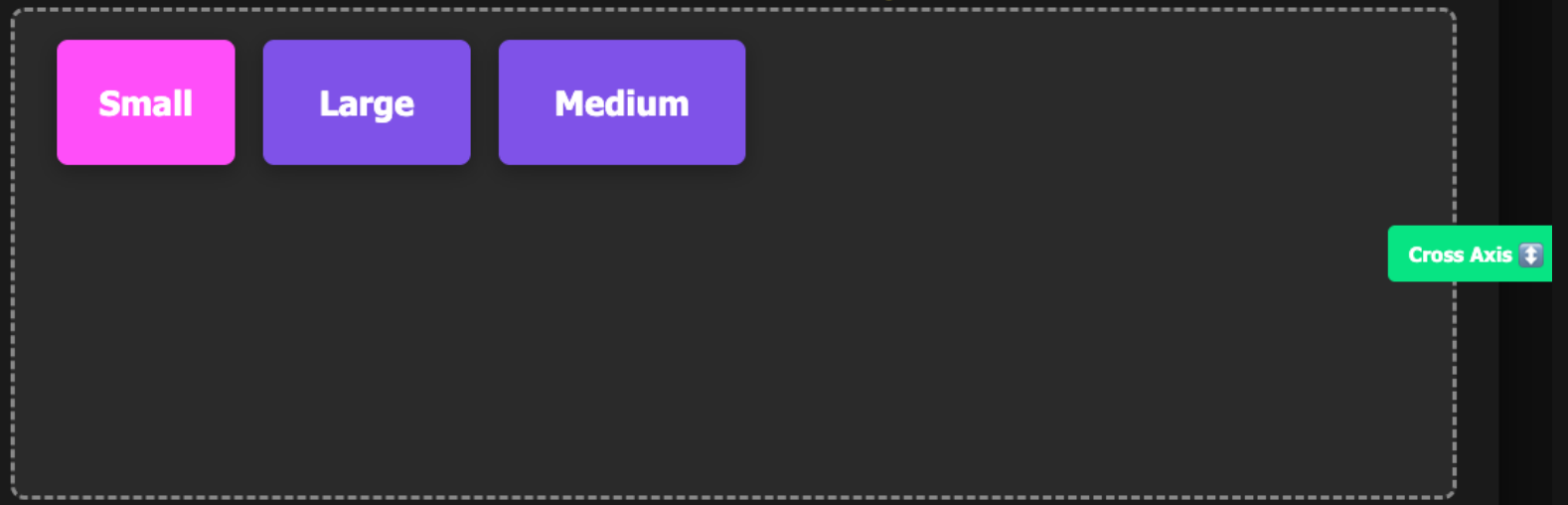
- ✓ 🌟 Vertical centering (THE SOLUTION!)
- ✓ 🌟 Hero sections
- ✓ 🌟 Modal dialogs
- ✓ 🌟 Buttons with icons
- ✓ 🌟 Navigation items

align-items (baseline)

baseline 📝

```
align-items: baseline; /* Text baseline alignment */
```

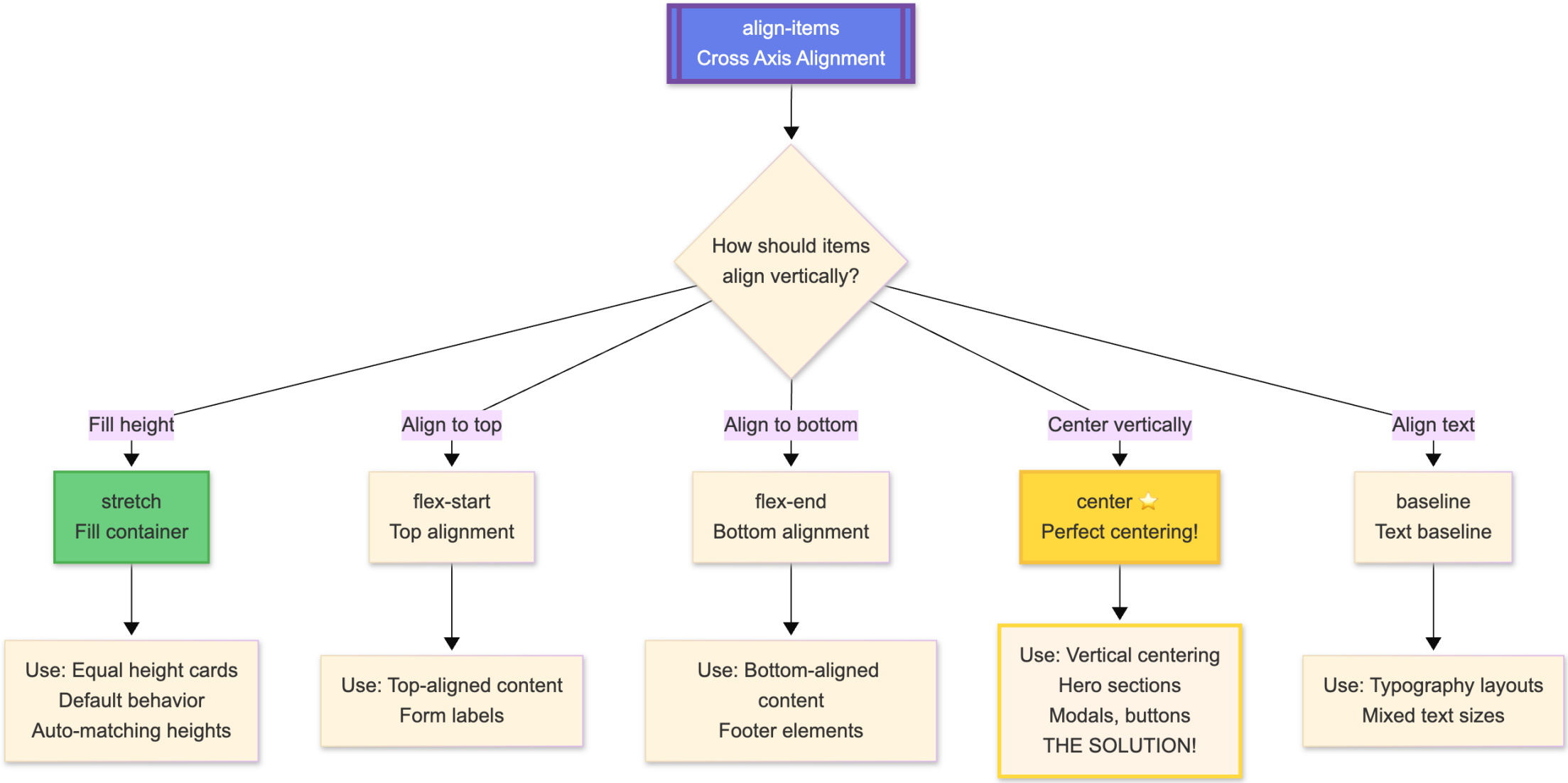
📝 Text Baselines Aligned



When to Use:

- ✓ Mixed text sizes
- ✓ Typography layouts
- ✓ Form inputs with labels
- ✓ Navigation with different font sizes

Align items Overview



Flex-wrap

Why This Matters:

- Makes layouts RESPONSIVE automatically!
- Desktop: Items in one row
- Mobile: Items wrap to multiple rows
- No media queries needed for basic responsiveness!

When to Use:

- Card layouts (Netflix-style rows that wrap)
- Image galleries (wrap on small screens)
- Image galleries (wrap on small screens)
- Tag clouds (tags wrap naturally)
- Product grids (items flow and wrap)

```
.container {
  flex-wrap: nowrap;        /* Default - squeeze/shrink items */
  flex-wrap: wrap;          /* Wrap to new line! */
  flex-wrap: wrap-reverse; /* Wrap upward (rare) */
}
```

```
nowrap (default) - Items squeeze to fit on one line
Wide Container:

┌──────────────────────────┐
│ [1] [2] [3] [4] [5]      │
└──────────────────────────┘

Narrow Container:

┌──────────────┐
│[1][2][3][4][5]│ ← Items shrink/overflow!
└──────────────┘

wrap - Items wrap to new rows when needed ⭐
Wide Container:

┌──────────────────────────┐
│ [1] [2] [3] [4] [5]      │
└──────────────────────────┘

Narrow Container:

┌──────────────┐
│ [1] [2] [3]  │ ← Wraps to new row!
│ [4] [5]      │
└──────────────┘

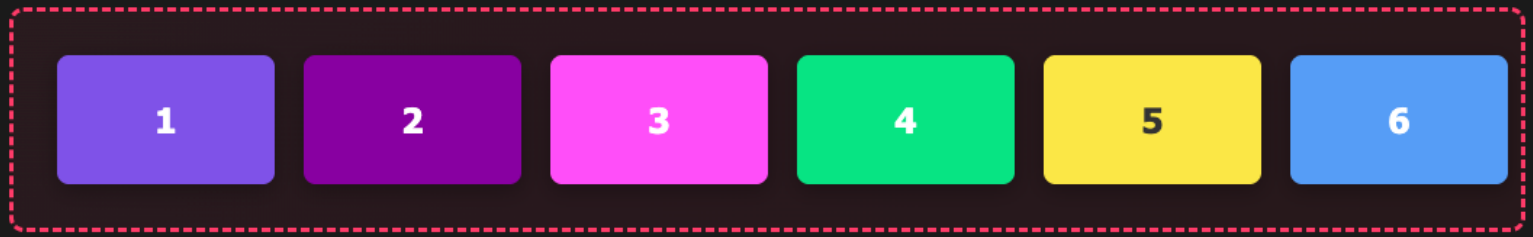
wrap-reverse - Wraps upward (bottom to top)
Narrow Container:

┌──────────────┐
│ [4] [5]      │ ← New rows appear ABOVE!
│ [1] [2] [3]  │
└──────────────┘
```

Flex-wrap (nowrap)

nowrap ✕ DEFAULT

```
flex-wrap: nowrap; /* Default - Items shrink/overflow */
```



What Happens:

- ✕ Items **squeeze** to fit on one line
- ✕ Items get **smaller** as container shrinks
- ✕ May **overflow** container
- ✕ Poor **mobile experience**

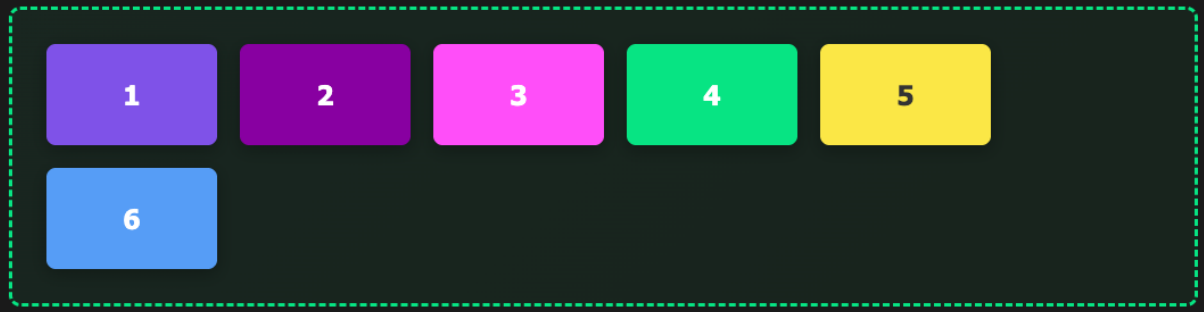
When to Use:

- ✓ Navigation bars (stay on one line)
- ✓ Single row toolbars
- ✓ When wrapping is unwanted

Flex-wrap (wrap)

wrap ✅ RECOMMENDED

```
flex-wrap: wrap; /* ✅ Items wrap to new row! */
```



What Happens:

- ✅ Items **wrap** to new rows when needed
- ✅ Items **maintain their size**
- ✅ **Automatic responsive** behavior
- ✅ **No media queries** needed!

Perfect For (MOST COMMON!):

- ✓ 🌟 Card layouts (Netflix-style rows)
- ✓ 🌟 Image galleries (wrap on small screens)
- ✓ 🌟 Product grids (items flow naturally)
- ✓ 🌟 Tag clouds (tags wrap automatically)
- ✓ 🌟 Dashboard widgets
- ✓ 🌟 Button groups

Flex-wrap (wrap-reverse)

wrap-reverse RARE

```
flex-wrap: wrap-reverse; /* Wraps upward (rare) */
```



What Happens:

- Items wrap **upward** (bottom to top)
- New rows appear **ABOVE** instead of below
- Visual order **reverses**

When to Use:

- ✓ Creative effects
- ✓ Reverse timelines
- ✓ Chat interfaces (rare)
- ✓ Specific design requirements

Why flex-wrap: wrap is AMAZING

Automatic Responsive

Desktop: one row. Mobile: multiple rows. **Automatically adapts!**

Mobile-Friendly

Items maintain readable sizes on small screens instead of getting squished

No Media Queries

For basic wrapping, you don't need complex breakpoints. It **just works!**

Natural Flow

Items flow like text - wrap when space runs out, unwrap when space available

One Line of CSS

Add `flex-wrap: wrap` and boom - responsive!

Perfect for Grids

Card layouts, galleries, products - anything that needs to wrap naturally

Gap Property

Why This Matters:

- Cleaner code - One property instead of margin tricks
- Only affects SPACE BETWEEN items (no edge margins!)
- Works in both directions (row and column)
- No negative margin hacks needed
- More intuitive - "I want 20px gap between items"

No more margin hacks! Cleaner, simpler, better!

```
/* Messy! 😫 */
.item {
  margin-right: 20px;
  margin-bottom: 20px;
}
.item:last-child {
  margin-right: 0; /* Remove margin from last item */
}
/* Gross calculations, edge cases everywhere! */

/* Clean! 😍 */
.container {
  display: flex;
  gap: 20px;        /* Space between ALL items */
  gap: 10px 20px;   /* row-gap column-gap */
}
/* No margin, no edge cases, just works! */
```

Gap property – Rows and Columns

Two Values for Different Directions

```
.container {
gap: 10px 30px; /* 10px rows, 30px columns */
}

/* Or use individual properties: */
.container {
row-gap: 10px; /* Vertical spacing */
column-gap: 30px; /* Horizontal spacing */
}
```

1 2 3 4 5 6 7 8

Notice: **10px vertical** gaps (between rows), **30px horizontal** gaps (between columns)!

Gap Property - Advantages

The Takeaway

- **Stop using margins for spacing flex items!**
- Use gap instead.
- It's **cleaner**, **simpler**, and **better**!
- No more margin hacks!

❌ The Old Way

MARGIN HACKS

```
/* Messy! 😫 */
.item {
margin-right: 20px;
margin-bottom: 20px;
}
.item:last-child {
margin-right: 0;
/* Remove last margin */
}
/* Gross calculations, edge cases! */
```



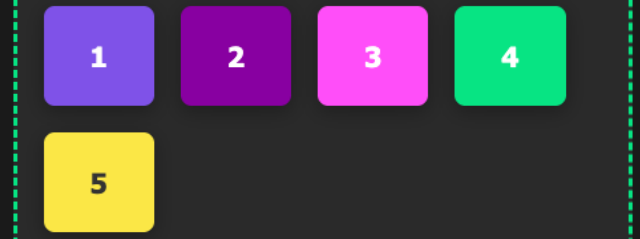
Problems:

- ❌ Multiple selectors needed
- ❌ Edge margins create issues
- ❌ :last-child hacks required
- ❌ Breaks with flex-wrap
- ❌ Complex calculations
- ❌ Hard to maintain

✅ The Modern Way

GAP PROPERTY

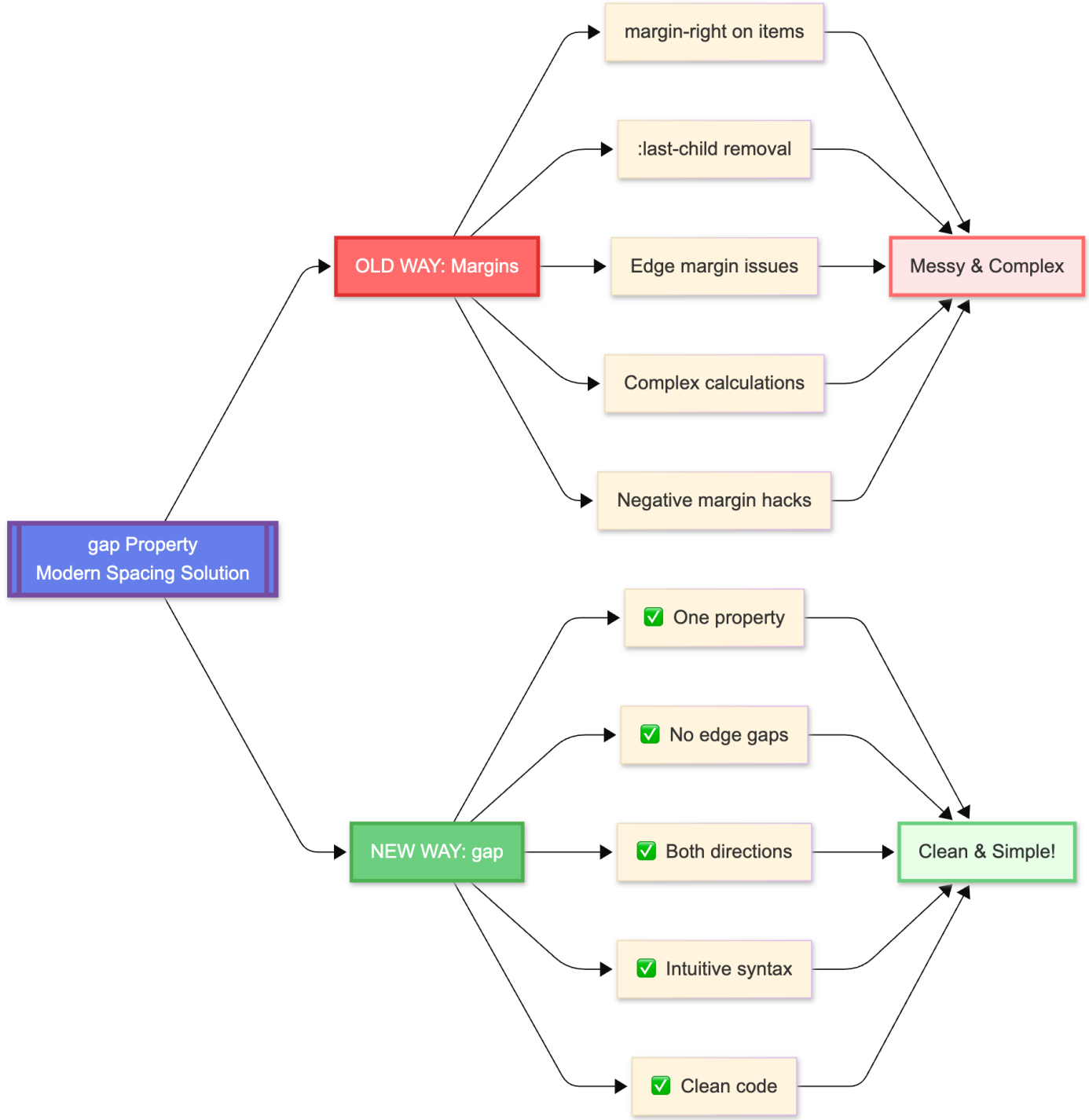
```
/* Clean! 😍 */
.container {
display: flex;
gap: 20px; /* DONE! */
}
/* No margin, no edge cases, just works! */
```



Benefits:

- ✅ One property - that's it!
- ✅ No edge gaps automatically
- ✅ No :last-child needed
- ✅ Works perfectly with wrap
- ✅ Intuitive and clean
- ✅ Easy to maintain

Modern Spacing Between Items!



ITEM PROPERTIES



Flex Item Properties Overview

1- flex-grow
- How much should this item GROW to fill space?
- Default: 0 (don't grow)

2- flex-shrink
- How much should this item SHRINK when space is tight?
- Default: 1 (can shrink)

3- flex-basis
- Starting size before growing/shrinking
- Like width, but flex-aware

4- flex (shorthand)
- Combines grow, shrink, and basis
- `flex: 1` = Most common usage!

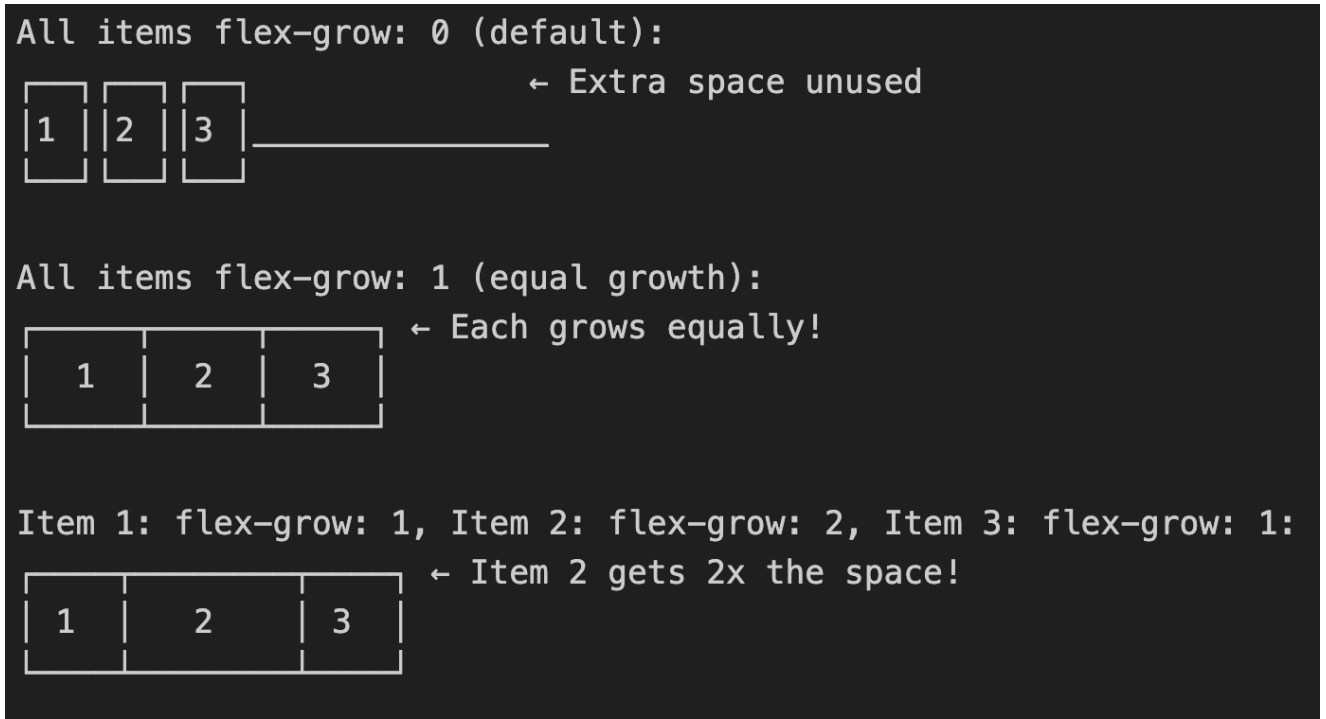
5- align-self
- Override container's align-items for THIS item
- Individual alignment control

Container = Team rules

Items = Individual player moves!

Flex-grow

```css
.item {
  flex-grow: 0;  /* Default - don't grow */
  flex-grow: 1;  /* Grow to fill available space */
  flex-grow: 2;  /* Grow 2x as much as flex-grow: 1 */
}
```



flex-grow: How Much Should This Item Grow?

How It Works:

- Available Space = Extra room after items fit at their natural size
- flex-grow divides that extra space proportionally!

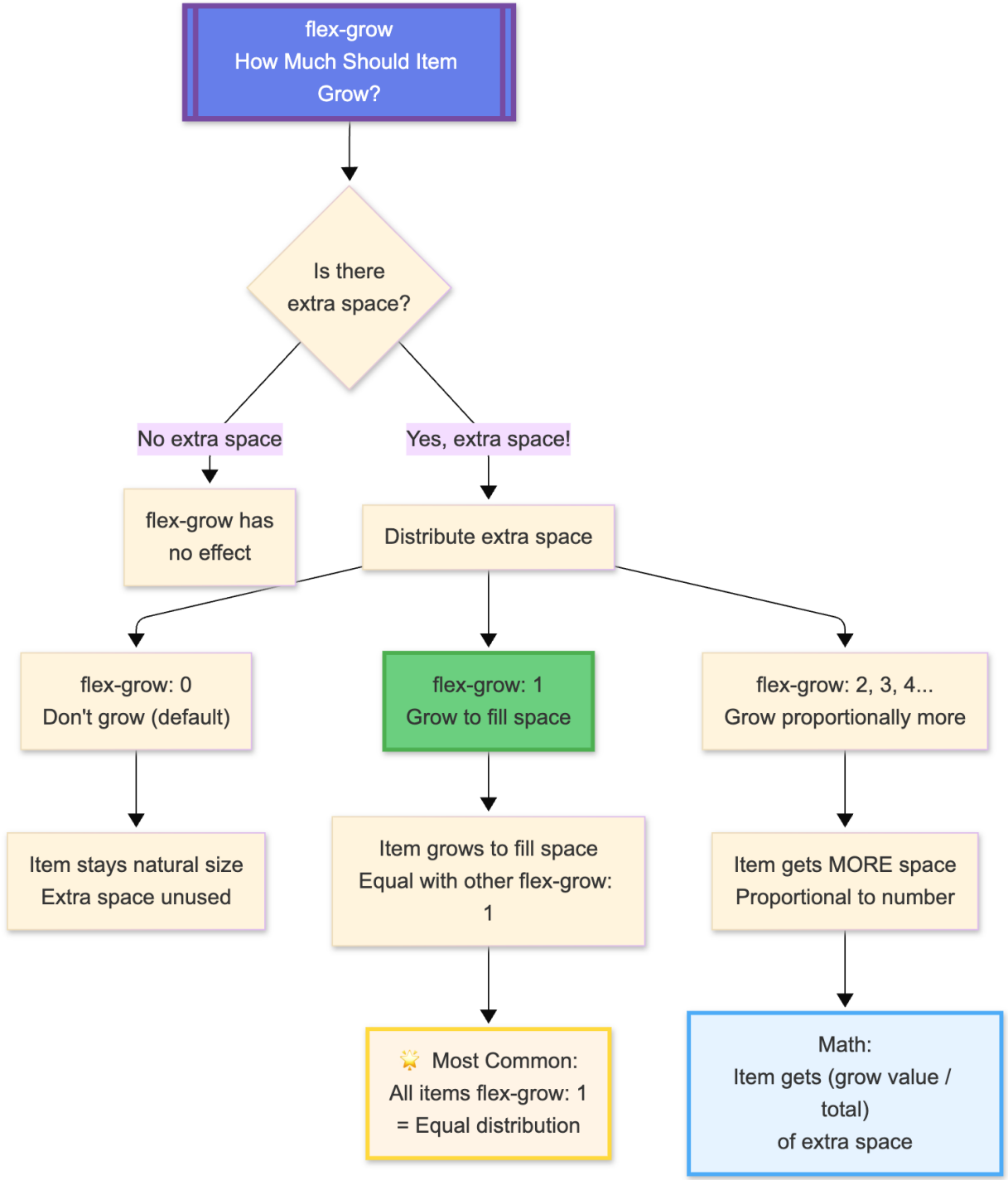
When to Use:

- Card layouts (Netflix-style rows that wrap)
- Image galleries (wrap on small screens)
- Tag clouds (tags wrap naturally)
- Product grids (items flow and wrap)

The Math:

- Total grow values: 1 + 2 + 1 = 4
- Item 1 gets: 1/4 of extra space (25%)
- Item 2 gets: 2/4 of extra space (50%)
- Item 3 gets: 1/4 of extra space (25%)

Flex-grow (How Much Should Item Grow?)



flex-grow Example

HTML:

```html
<div class="container">
    <div class="item fixed">Fixed Width</div>
    <div class="item grow-1">Grows 1x</div>
    <div class="item grow-2">Grows 2x (double!)</div>
</div>
```

Flex Container (Parent)

- Gray box (Fixed) - Stays at natural content size
- Green box (flex-grow: 1) - Grows to fill space
- Red box (flex-grow: 2) - Grows TWICE as much as green!

Container: 900px total width

- Fixed item: 100px (natural size)
- Remaining space: 800px

Distribution of 800px:

- Green (1): 800px × 1/3 = ~267px
- Red (2): 800px × 2/3 = ~533px ← Double!

CSS:

```css
.container {
  display: flex;
  gap: 10px;
  border: 2px solid #333;
}

.item {
  padding: 20px;
  background-color: #3498db;
  color: white;
}

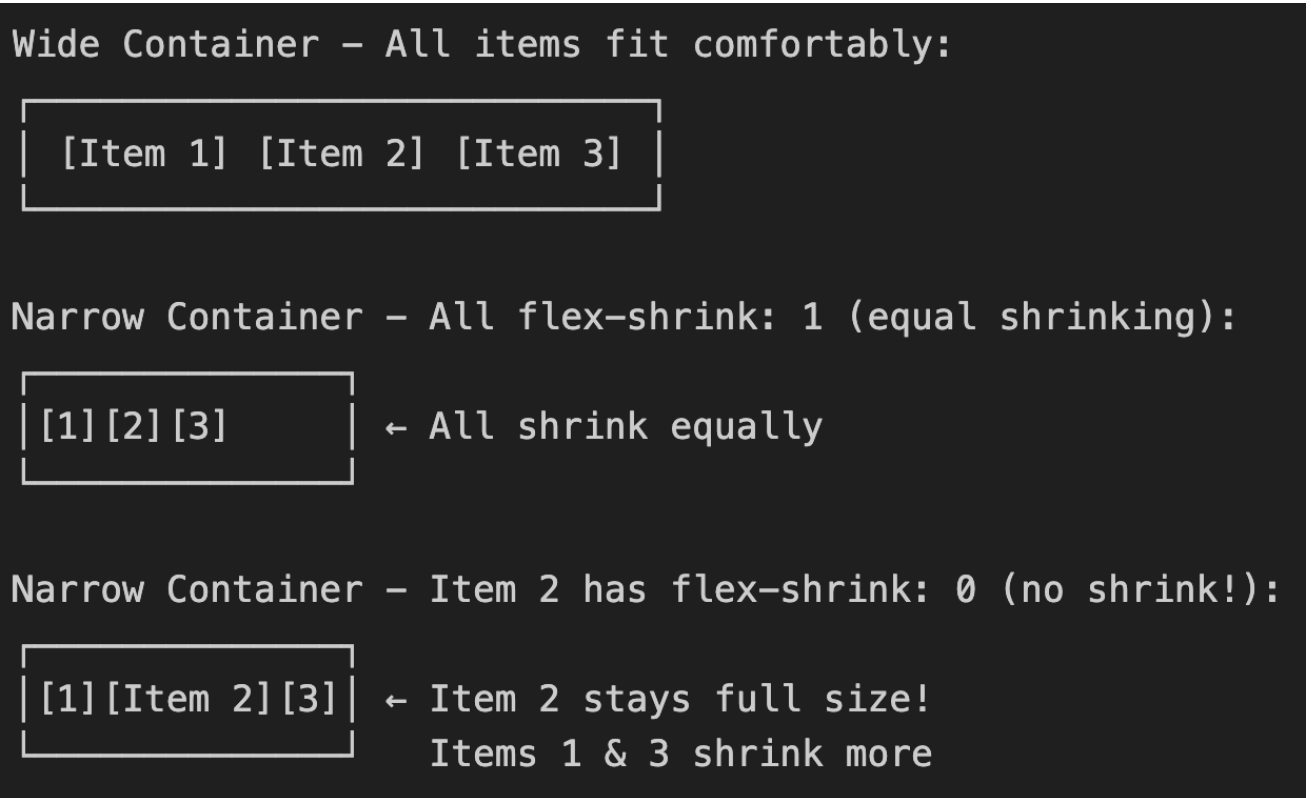
.fixed {
  /* No flex-grow - stays at natural size */
  background-color: #95a5a6;
}

.grow-1 {
  flex-grow: 1;  /* Gets 1 part of extra space */
  background-color: #2ecc71;
}

.grow-2 {
  flex-grow: 2;  /* Gets 2 parts = DOUBLE! */
  background-color: #e74c3c;
}
```

Flex-shrink

```css
.item {
  flex-shrink: 1;  /* Default - can shrink */
  flex-shrink: 0;  /* NEVER shrink - stay at size! */
  flex-shrink: 2;  /* Shrink 2x as much */
}
```



flex-shrink: How Much Should This Item Shrink?

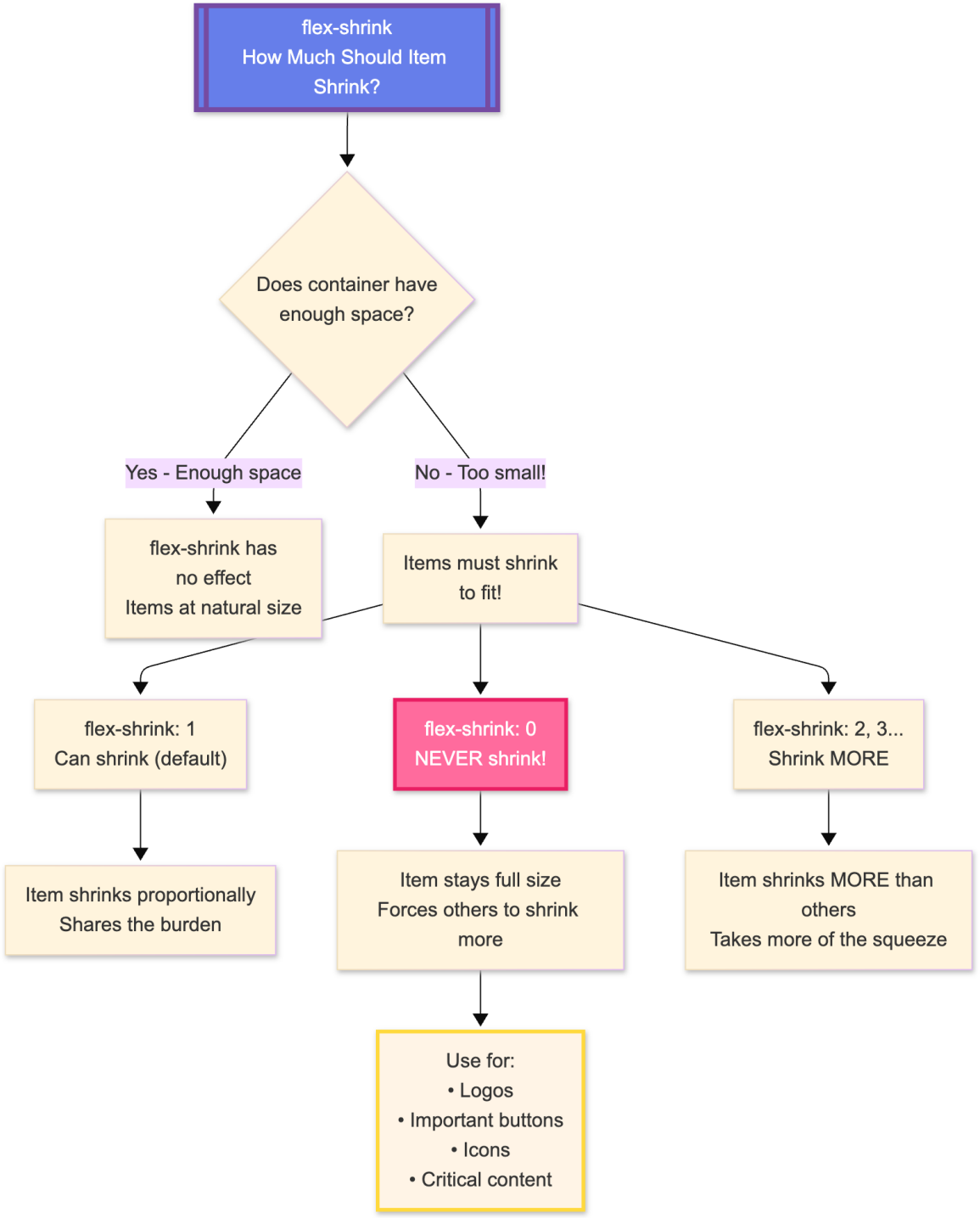
How It Works:

- When container is TOO SMALL for all items: flex-shrink determines which items shrink (and how much)!

Common Use Cases:

- flex-shrink: 1 (default) - Allow shrinking
- flex-shrink: 0 - Keep minimum size (important content!)
- Most common: Set to 0 for items that shouldn't shrink

Flex-shrink
How Much Should Item Shrink?



Flex-shrink – Common Use Cases

Keep Default (flex-shrink: 1)

```
.item {
flex-shrink: 1;
}
/* Default - can shrink */
```

Use for: Most items - allow flexible shrinking for responsive layouts

Never Shrink (flex-shrink: 0) ⭐

```
.logo {
flex-shrink: 0;
}
/* NEVER shrink */
```

Use for: Logos, important buttons, icons - must stay readable!

Navigation Header

```
.logo { flex-shrink: 0; }
.nav { flex-shrink: 1; }
.cta { flex-shrink: 0; }
```

Logo and CTA stay full size, navigation shrinks if needed

Form Layout

```
.label { flex-shrink: 0; }
.input { flex-shrink: 1; }
.button { flex-shrink: 0; }
```

Label and button stay readable, input field flexes

Toolbar with Icons

```
.icon { flex-shrink: 0; }
.text { flex-shrink: 1; }
```

Icons stay visible, text can truncate if needed

Card with Image

```
.image { flex-shrink: 0; }
.content { flex-shrink: 1; }
```

Image maintains aspect ratio, content flexes

Flex-basis

flex-basis: Starting Size Before Flex!

What It Does:

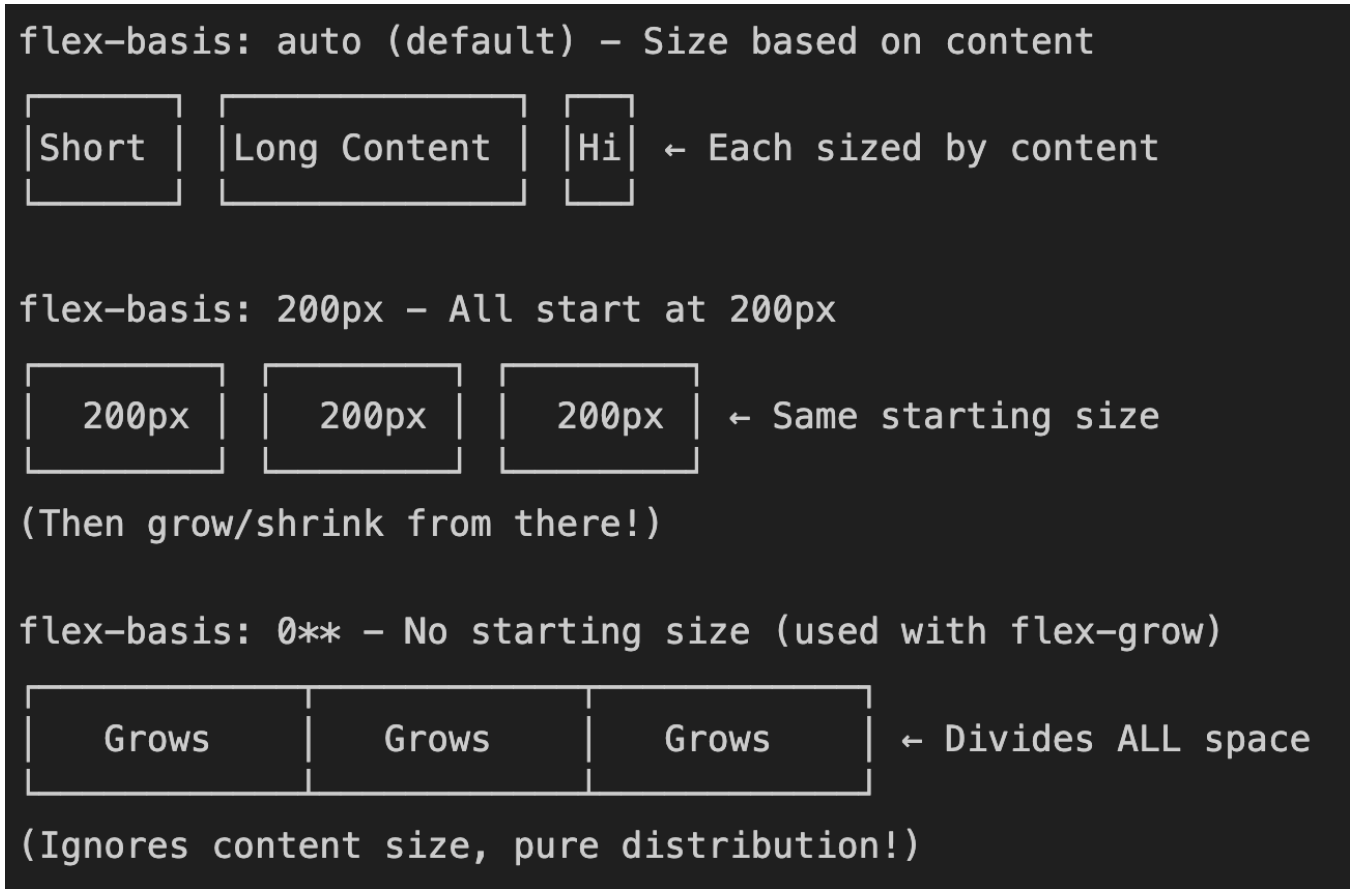
- Sets the **STARTING size** before growing or shrinking
- Like width, but **flex-aware** and better!
- Works on **main axis** (width for row, height for column)
- More flexible than fixed width

Common Values:

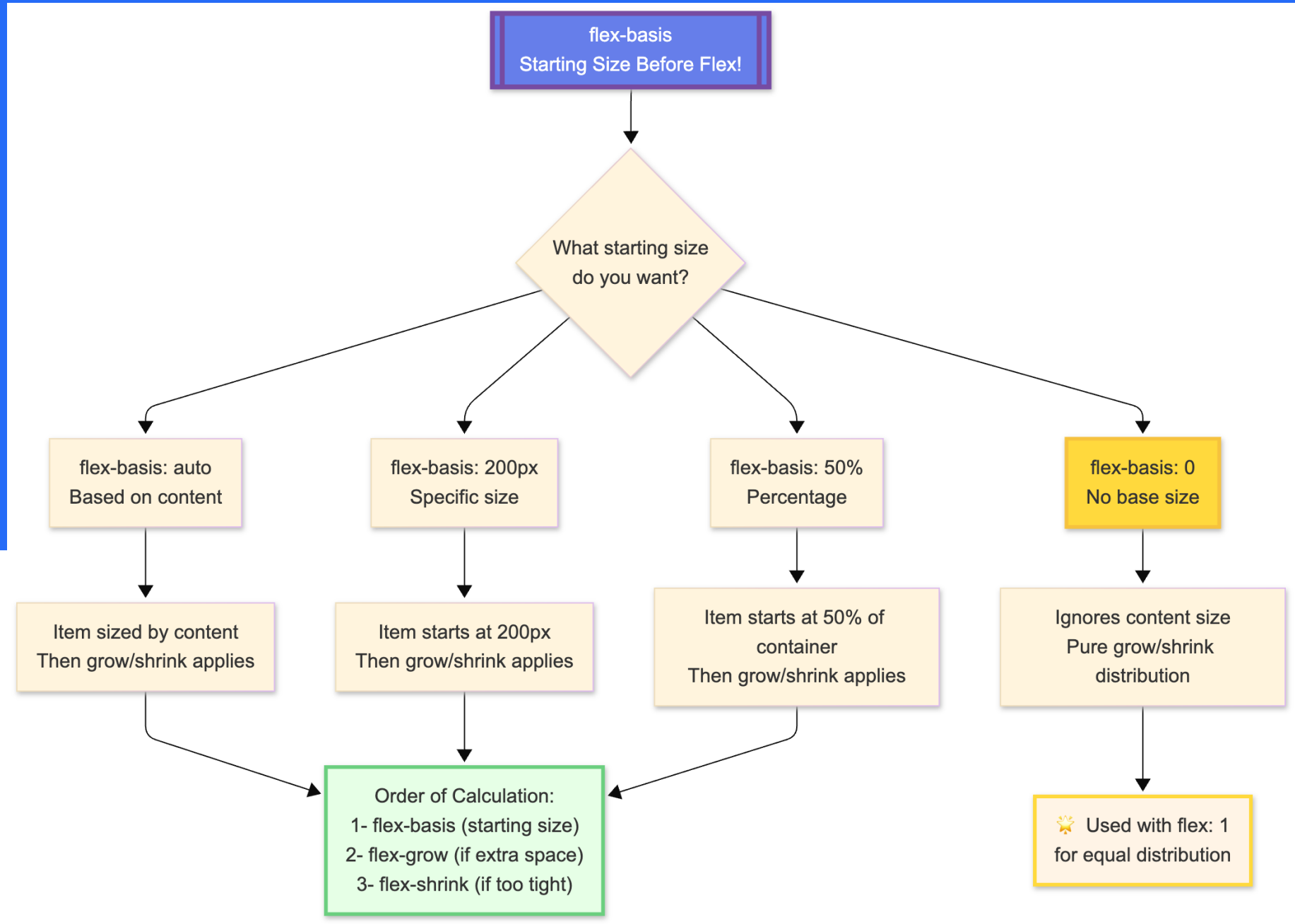
- auto - Size based on content (default)
- 200px / 20% - Specific starting size
- 0 - Ignore content, divide space by grow values

Think: What size do I want this BEFORE it flexes?

```css
.item {
  flex-basis: auto;  /* Default - based on content */
  flex-basis: 200px; /* Start at 200px */
  flex-basis: 50%;   /* Start at 50% of container */
  flex-basis: 0;     /* No base size (used with flex: 1) */
}
```



Flex-basis Starting Size Before Flex!



Flex-basis Quick Reference Table

Value	Behavior	When to Use
`auto`	Size based on content (default)	Natural, content-driven layouts
`200px`	Start at 200px, then grow/shrink	Controlled starting size
`0`	No base size, pure distribution	Equal distribution with `flex: 1`
`50%`	50% of container width	Responsive percentage sizing
`min-content`	Smallest possible size	Compact layouts
`max-content`	Largest needed size	Content-fits layouts

Flex Shorthand

flex: The Shorthand Property - USE THIS!

Why Use Shorthand?

- Cleaner, shorter code
- Sets all three values at once
- flex: 1 is easier than three separate properties
- More maintainable
- Industry standard

Remember:
flex: 1 = "Give me equal space!"

```
.item {
  flex: 1;  /* Most common - you'll use this constantly! */

  /* Full syntax:
     flex: <flex-grow> <flex-shrink> <flex-basis>;
  */
  flex: 1 1 0%;  /* What flex: 1 actually means */
}
```

```
/* Common Patterns You'll Use: */
/* flex: 1 MOST COMMON - Equal space distribution */
.item {
  flex: 1;  /* Same as: flex-grow: 1, flex-shrink: 1, flex-basis: 0 */
}
/* Result: All items grow equally, ignore content size, divide space evenly! */

/* flex: 0 0 200px - Fixed size, no flex */
.item {
  flex: 0 0 200px;  /* Don't grow, don't shrink, stay 200px */
}
/* Result: Rigid 200px width, doesn't change! */

/* flex: 0 1 auto - Default behavior */
.item {
  flex: 0 1 auto;  /* Don't grow, can shrink, content-based size */
}
/* Result: Natural content size, can shrink if needed */
```

Flex - Quick Reference Table

Shorthand	Expands To	Result	Use Case
`flex: 1` ⭐	`1 1 0`	Equal distribution	Most common - equal columns
`flex: auto`	`1 1 auto`	Content-based + flexible	Variable content
`flex: none`	`0 0 auto`	No flexing	Fixed content size
`flex: 0 0 200px`	`0 0 200px`	Fixed 200px	Sidebars, fixed columns
`flex: 0 1 auto`	`0 1 auto`	Content size, can shrink	Default behavior

flex Shorthand - Example

HTML:

```html
<section class="container">
    <div class="box">1</div>
    <div class="box">2</div>
    <div class="box">3</div>
    <div class="box">4</div>
    <div class="box">5</div>
    <div class="box">6</div>
</section>
```

CSS:

```css
.container {
    display: flex;
    gap: 20px;
}

.column {
    /* MOST COMMON - Equal space distribution */
    flex: 1;
    /* Fixed size, no flex */
    flex: 0 0 200px;
    /* Default behavior */
    flex: 0 1 auto;

    padding: 10px;
    border: 1px solid #ccc;
    background-color: #3498db;
}
```

Remember: flex: 1 = "Give me equal space!"

Alig-self

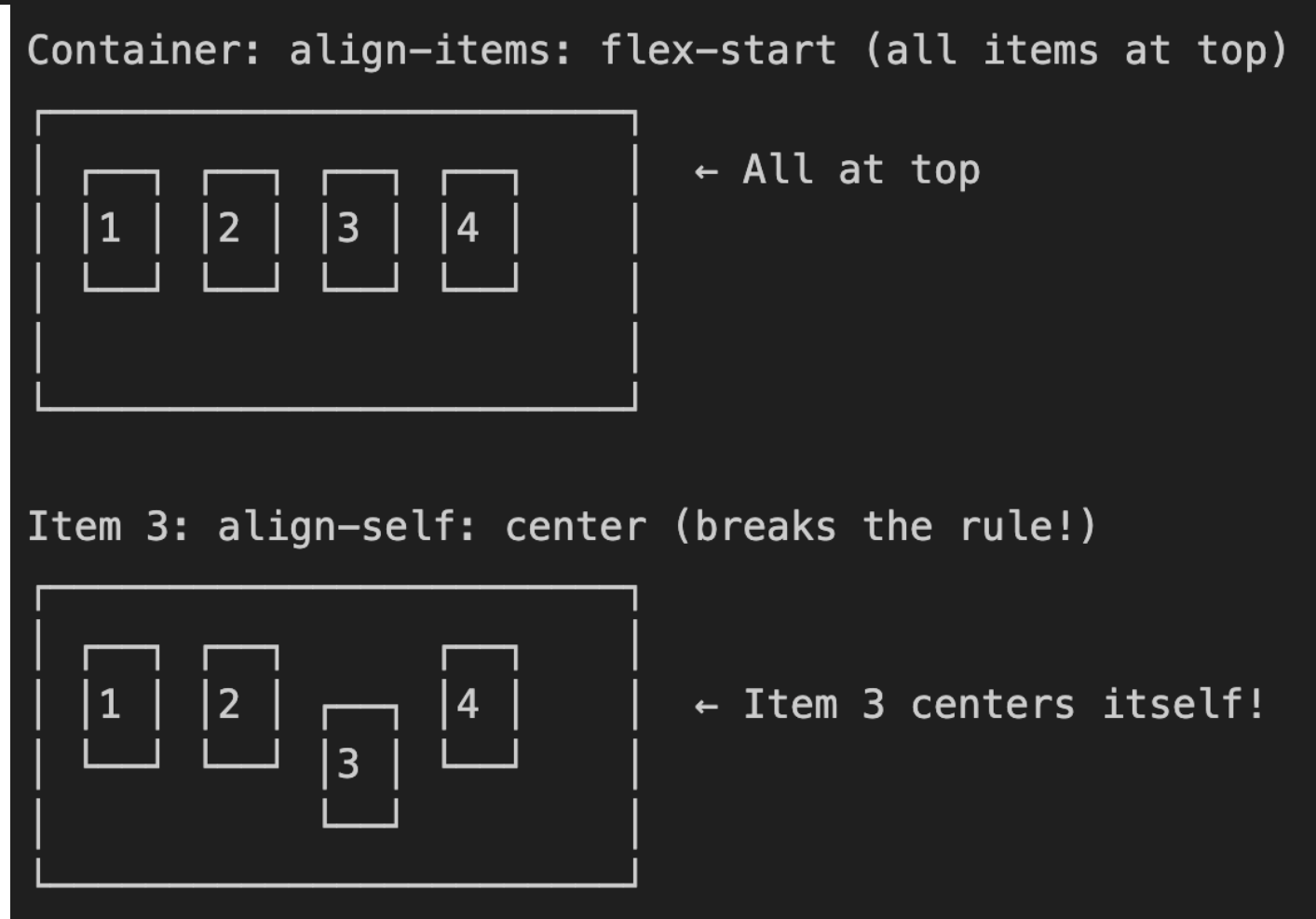
```css
.item {
  align-self: auto;         /* Inherit from container (default) */
  align-self: flex-start;   /* Align to start of cross axis */
  align-self: flex-end;     /* Align to end of cross axis */
  align-self: center;       /* Center on cross axis */
  align-self: stretch;      /* Fill container height */
  align-self: baseline;     /* Align text baseline */
}
```

align-self gives individual items the power to override the container's align-items rule.

When to Use:

- Featured/special items
- Icons with text
- One item needs different alignment
- Don't overuse - most items should follow container

Individual control! Break the container rule when needed!




Flexbox concepts?
Item properties?
Container properties?
Building navigation bars?
Anything from this week?
Anything unclear?
HAVE ANY
QUESTIONS?